TECHNICAL PORTFOLIO CASE STUDY

Transportation Exception Management API

Runtime validation, transportation workflow modelling, persistence, reporting, testing, and CI

Project type	Independent C#/.NET portfolio project
Validation date	11 August 2026 - Windows
Evidence	10 public operations, SQLite persistence, 36 integration tests
Data boundary	Deterministic synthetic transportation data only

SUPPLY CHAIN -> ANALYTICS -> AUTOMATION -> SYSTEMS

Not an Amazon application, employer system, commercial platform, or production deployment. No real carrier or customer data.

Executive summary

Transportation Exception Management API models how delayed, constrained, disrupted, or documentation-blocked movements can be recorded, assigned, progressed, explained, reported, and exported through a consistent backend workflow. The solution uses ASP.NET Core, application services, a domain state machine, EF Core, and SQLite. It is deliberately self-contained and reproducible.

The validation session exercised every public operation in the existing Swagger UI. A new synthetic case was created, rejected from active work while unassigned, assigned, noted, transitioned, re-read, included in aggregate reports, and exported to CSV. Read-only SQLite inspection confirmed the migration, relationship, case row, and note row. The automated suite passed all 36 tests.

Runtime surface	Persistence	Automated tests	Remote CI
10 operations tested	37 cases / 13 notes	36 passed / 0 failed	Latest checked run: success

Evidence boundary

All cases, movements, nodes, carriers, analysts, timestamps, due-time thresholds, and report values are fabricated. The project demonstrates engineering discipline without claiming professional production C#/.NET experience or real-world business impact.

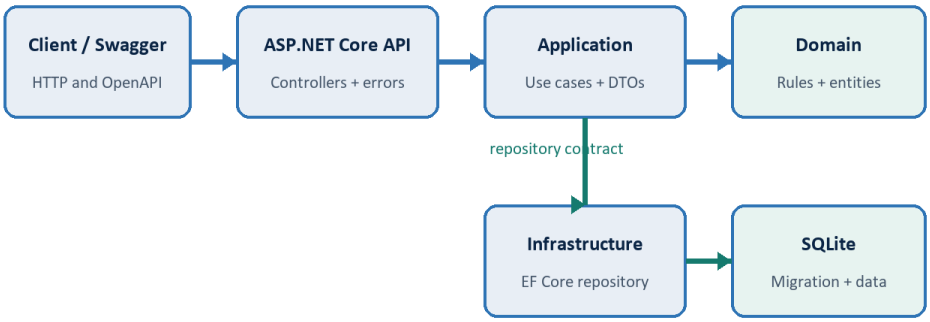
What the project proves

- Transportation-domain concepts translated into explicit entities, lifecycle rules, and due-time reporting.
- A layered .NET solution whose project references match its claimed dependency boundaries.
- Relational persistence, migrations, deterministic seed data, API validation, tests, and CI working together.
- Limitations and synthetic-data boundaries documented as carefully as successful behaviours.

System architecture

The domain is the dependency root; API is the composition root; Infrastructure implements the Application repository contract.

Actual runtime and dependency flow



Domain has no HTTP or EF Core dependency. API is the composition root.

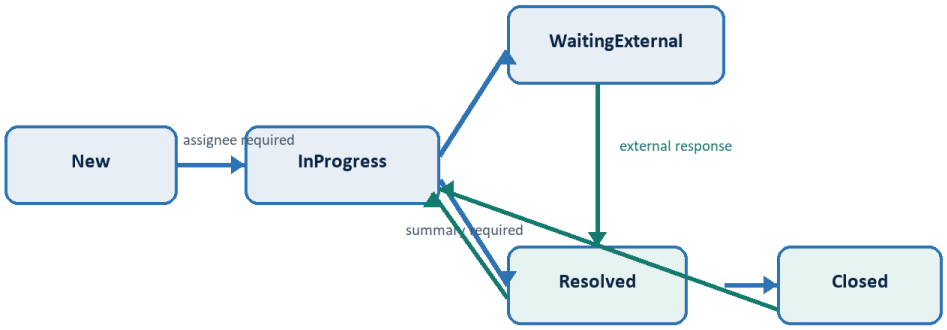
Layer	Confirmed responsibility
Domain	Entities, enums, invariants, status graph, field limits, and illustrative SLA policy.
Application	DTOs, queries, services, reports, CSV export, mappings, and ICaseRepository.
Infrastructure	EF Core context and mappings, SQLite repository, migration, initializer, and seed.
API	Controllers, dependency composition, validation, ProblemDetails, health, and Swagger.
Tests	Hosted HTTP pipeline, in-memory SQLite, fixed clock, and migration-backed scenarios.

The design intentionally avoids MediatR, AutoMapper, and generic-repository ceremony. For this single-service demonstration, direct application services and an explicit repository contract keep the important rules visible.

Transportation domain and lifecycle

A case connects a fictional movement, route nodes, carrier code, exception category, severity, status, ownership, timestamps, description, resolution data, and notes. Origin and destination must differ; assignment is required before active work; resolution requires a summary; reopening clears stale resolution fields.

Illustrative transportation case lifecycle



Only these edges are valid; other requests return HTTP 409.

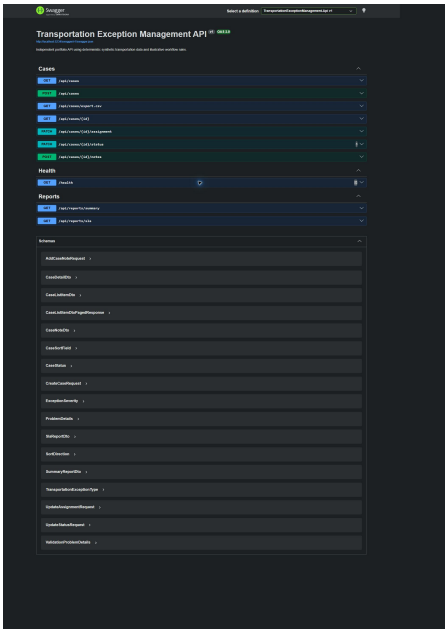
Severity	Illustrative target	Purpose
Critical	2 hours	Highest synthetic urgency
High	4 hours	Validation case severity
Medium	8 hours	Moderate synthetic urgency
Low	24 hours	Lowest synthetic urgency

These thresholds are invented portfolio rules, not carrier commitments or operational standards.

API surface

Method	Route	Purpose
GET	/health	Process health
GET	/api/cases	Filtered, sorted, paginated list
POST	/api/cases	Validated case creation
GET	/api/cases/export.csv	Filtered CSV export
GET	/api/cases/{id}	Detail and notes
PATCH	/api/cases/{id}/assignment	Set ownership
PATCH	/api/cases/{id}/status	Validated lifecycle command
POST	/api/cases/{id}/notes	Append context
GET	/api/reports/summary	Workload aggregates
GET	/api/reports/sla	Illustrative due-time outcomes

Swagger contract overview

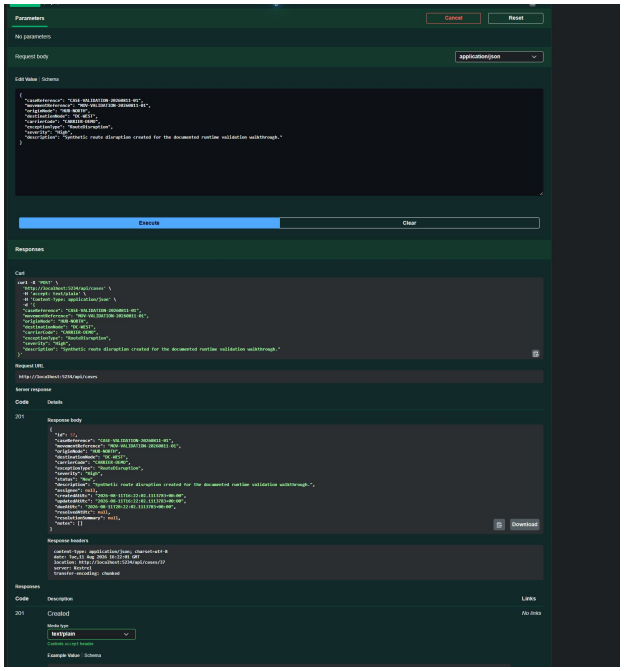


The live OpenAPI surface matched the controllers: ten operations across Cases, Health, and Reports.

Runtime workflow - case 37

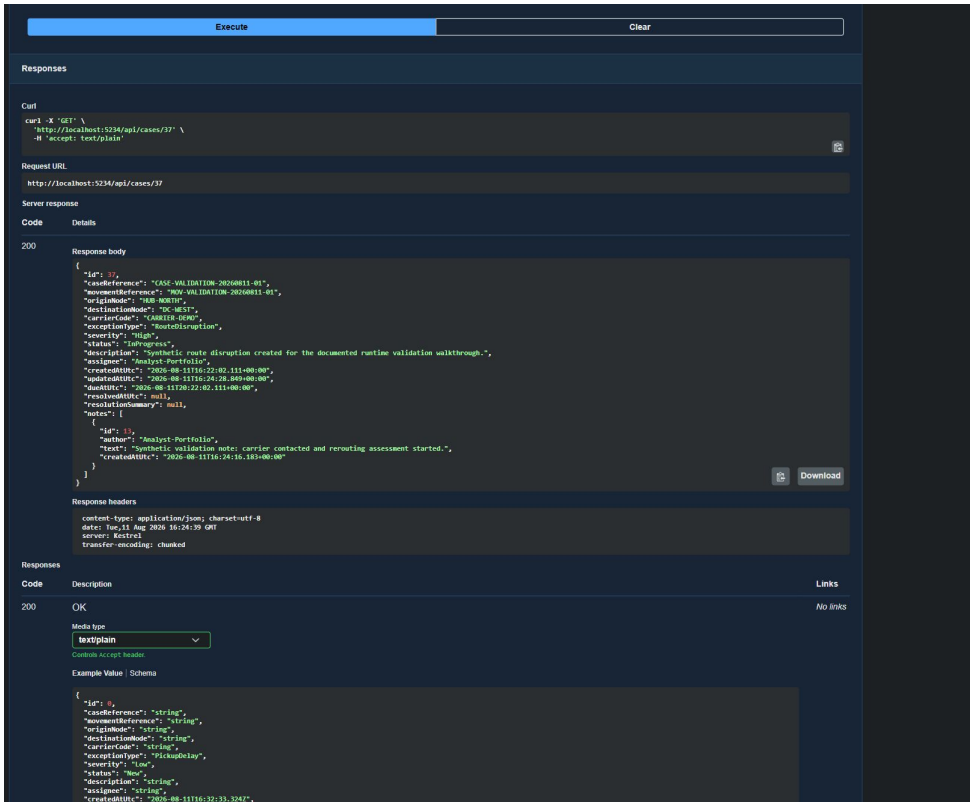
The walkthrough used CASE-VALIDATION-20260811-01 and fabricated values only.

Step	Request	Status	Evidence
1	GET /health	200	Healthy
2	GET /api/cases?pageSize=3	200	36 seeded cases before mutation
3	POST /api/cases	201	ID 37, New, four-hour High due time
4	PATCH status -> InProgress	409	assignee_required
5	PATCH assignment	200	Analyst-Portfolio
6	POST note	201	Note 13
7	PATCH status -> InProgress	200	Valid edge
8	GET /api/cases/37	200	Assignee, status, and note persisted
9	PATCH status -> Closed	409	invalid_transition; no mutation

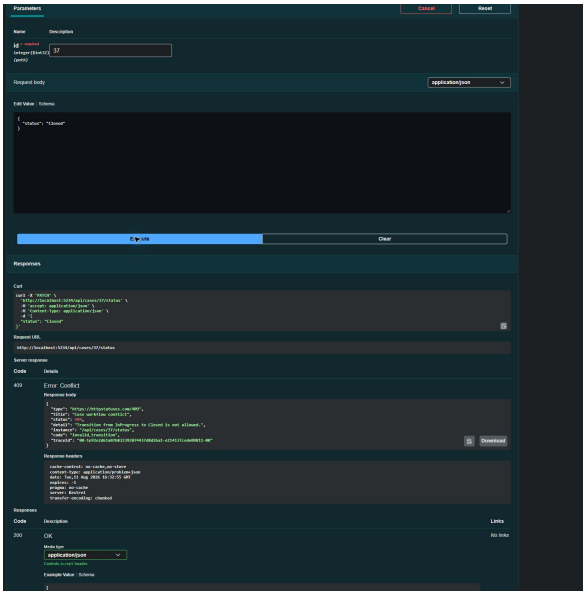


Swagger returned 201 Created, a detail Location header, New status, and the calculated due time.

Persistence and workflow evidence



A later GET returned the persisted InProgress state, Analyst-Portfolio ownership, and note 13.



InProgress -> Closed was rejected with invalid_transition; a later GET still showed InProgress.

Database and EF Core validation

Evidence	Observed result
Database	SQLite file configured through DefaultConnection
Migration	20260810125518_InitialCreate; EF product version 10.0.10
Tables	TransportationExceptionCases, CaseNotes, EF migration history
Relationship	CaseNotes -> cases; required FK; ON DELETE CASCADE
Indexes	Unique case reference plus status, severity, type, assignee, and timestamps
Post-walkthrough	37 cases, 13 notes, case 37 and note 13 present

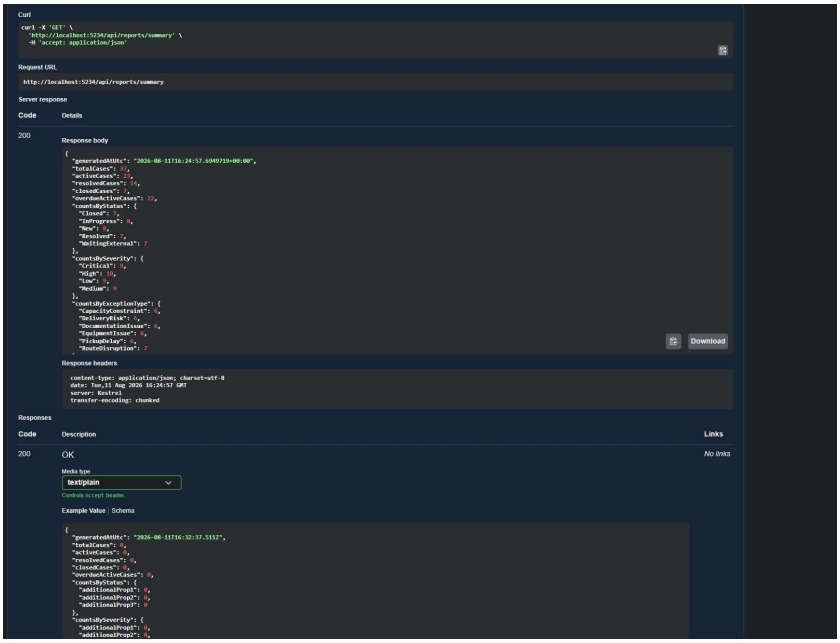
Startup sequence

- Create a scoped TransportationExceptionsDbContext.
- Call Database.MigrateAsync so the committed migration is the schema source of truth.
- Run SyntheticDataSeeder only after migration.
- Exit without inserting when any case already exists; otherwise add exactly 36 deterministic cases.

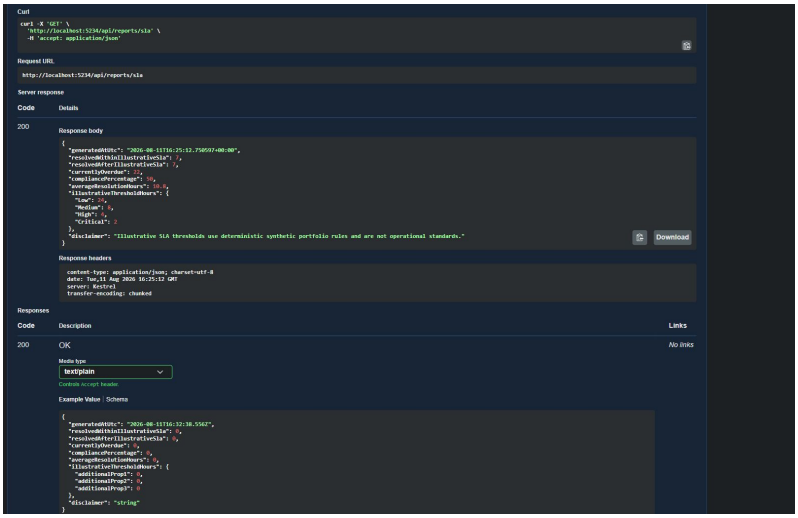
EF Core stores enums as readable strings. UTC DateTimeOffset values are converted to Unix-millisecond integers for consistent SQLite comparison and sorting, then materialized back to UTC.

Public documentation deliberately omits the machine-specific absolute database path. Database, WAL, and shared-memory files are ignored by Git.

Reporting and export



Summary after mutation: 37 total, 23 active, 14 resolved, 7 closed, 22 overdue active.



Illustrative SLA report: 7 within, 7 after, 50% compliance, 10.8 average resolution hours.

The filtered CSV response contained the 12-column header and one Analyst-Portfolio row. Export uses invariant timestamps, deterministic ordering, correct CSV quoting, and a prefix for formula-like values.

Testing strategy

The 36-test xUnit suite uses WebApplicationFactory to exercise the real HTTP pipeline. It replaces file-based persistence with one open SQLite in-memory connection and replaces TimeProvider.System with a fixed clock. The boundary therefore includes request binding, controllers, services, domain rules, EF Core translation, migrations, and relational persistence.

Representative scenario	Behaviour protected
Creation and due time	201 + Location + New + Critical two-hour due time
Assignment guard	Unassigned New cannot enter InProgress
Resolve / close / reopen	Summary required; resolution retained or cleared correctly
Notes	201 note is returned by a later detail GET
Reports	Totals and grouped dimensions reconcile to 36
Migration startup	Initial migration applied; seed exactly 36; no duplicate seed

AUTOMATED RESULT **Passed 36 | Failed 0 | Skipped 0 | Total 36**

Representative ownership test

```
var response = await ChangeStatusAsync(created.Id, CaseStatus.InProgress);

Assert.Equal(HttpStatusCode.Conflict, response.StatusCode);
Assert.Equal("assignee_required", await response.ReadProblemCodeAsync());
```

This is observable HTTP behaviour: the stable conflict code is asserted through the hosted API, not by calling the entity directly.

Selected implementation highlights

Domain transition graph

```
private static bool IsTransitionAllowed(CaseStatus current, CaseStatus requested) =>
    (current, requested) switch
    {
        (CaseStatus.New, CaseStatus.InProgress) => true,
        (CaseStatus.InProgress, CaseStatus.WaitingExternal or CaseStatus.Resolved) => true,
        (CaseStatus.WaitingExternal, CaseStatus.InProgress or CaseStatus.Resolved) => true,
        (CaseStatus.Resolved, CaseStatus.Closed or CaseStatus.InProgress) => true,
        (CaseStatus.Closed, CaseStatus.InProgress) => true,
        _ => false,
    };
```

The entity owns the lifecycle graph, keeping it independent of HTTP and EF Core.

SLA calculation

```
var resolved = cases.Where(item => item.ResolvedAtUtc.HasValue).ToArray();
var within = resolved.Count(item => item.ResolvedAtUtc <= item.DueAtUtc);
var after = resolved.Length - within;
decimal? compliance = resolved.Length == 0
    ? null
    : Math.Round(within * 100m / resolved.Length, 2);
```

Empty datasets return null rather than a misleading zero percentage, and resolved outcomes remain separate from active overdue work.

CSV defensive escaping

```
if (value.Length > 0 && value[0] is '=' or '+' or '-' or '@')
    value = $"'{value}";

if (value.Contains(',') || value.Contains('"') ||
    value.Contains('\r') || value.Contains('\n'))
    return $"\"{value.Replace("\"", "\"\"")}\"";
```

The exporter handles CSV quoting and reduces formula interpretation risk for spreadsheet consumers.

CI and engineering discipline

CI stage	Defined behaviour
Triggers	All pull requests and pushes to main
SDK	actions/setup-dotnet@v5 using global.json
Quality	Restore; dotnet format --verify-no-changes
Build / test	Release build and tests without duplicate work
Smoke	Temporary SQLite; bounded /health and OpenAPI checks; cleanup trap
Latest verified	Run #2 on commit 8710758 completed successfully

Engineering choices

- Layered but compact: clear dependency direction without framework ceremony.
- SQLite: relational realism and reproducibility without an external database server.
- DTO public contract: persistence entities do not leak into HTTP responses.
- String enums and ProblemDetails: readable Swagger and machine-readable errors.
- Fixed TimeProvider in tests: predictable due times and lifecycle assertions.
- Warnings as errors and deterministic builds configured centrally.

Remote status source: github.com/Rahulnanduri/transportation-exception-management-api/actions/runs/31391980858

Limitations and issue found

- No authentication, authorization, or authenticated audit identity.
- No optimistic concurrency for simultaneous editors.
- No carrier/customer integration, EDI, telematics, message broker, or event stream.
- No notifications, background workers, cloud deployment, or infrastructure as code.
- No production observability stack, high availability, or scale claim.
- No frontend beyond Swagger and no real-world SLA validation.

Runtime API-description issue

Observed	Swagger lists text/plain first for many JSON responses, so its default Accept header can make exception handling fall back to a generic problem body.
Expected path	Selecting application/json returns detail, instance, and the stable codes assignee_required or invalid_transition.
Action	Documented only. Application source code was not modified during this task.

The issue does not invalidate the workflow rule. The live request with application/json returned 409 and the correct stable code, and the automated test suite asserts the same response contract.

Run, test, and inspect

Run locally

```
git clone https://github.com/Rahulnanduri/transportation-exception-management-api.git
cd transportation-exception-management-api
dotnet tool restore
dotnet restore TransportationExceptionManagement.sln
dotnet run --project src/TransportationExceptionManagement.Api
```

Test locally

```
dotnet clean TransportationExceptionManagement.sln
dotnet build TransportationExceptionManagement.sln
dotnet test TransportationExceptionManagement.sln --no-build
```

Swagger: http://localhost:5234/swagger/index.html
OpenAPI: http://localhost:5234/swagger/v1/swagger.json

What this portfolio evidence communicates

Audience	Signal
Recruiter	A functioning independent project with clear stack, tests, and evidence.
Supply-chain hiring manager	Ownership, lifecycle, exception context, due-time reasoning, reports, and export.
Technical interviewer	Domain rules, project dependencies, EF migrations, relational tests, runtime errors, and honest limits.

SUPPLY CHAIN	ANALYTICS	AUTOMATION	SYSTEMS

Independent engineering evidence. Deterministic synthetic data. Illustrative workflow rules. No production or employer claim.